

实验一 选择器和编码器

2025 年秋季学期

To be, or not to be, that is the question.

– 《哈姆雷特》，莎士比亚

组合逻辑电路是数字电路的基础部件，组合逻辑电路可以抽象为将数字输入映射到数字输出的逻辑函数。不同于需要时钟控制的时序逻辑电路，组合逻辑无需时钟且内部没有记忆状态，当输入变化时输出直接发生对应变化。

选择器是、译码器和编码器是数字逻辑系统的常用电路，也是组合逻辑电路中的主要组成元件。

选择器是由多路数据输入、一位或多位的选择控制端，和一路数据输出所组成的。多路选择器从多路输入中，选取其中一路将其传送到输出端，由选择控制信号决定输出的是第几路输入信号。数字电路中存在大量的并行运算，通常的设计思路是使用电路同时生成所有可能用到的数字信号，再利用选择器选择最终输出哪一路信号。

译码器是将某一输入信息转换为某一特定输出的逻辑电路，通常是多路输入/输出电路，它将 n 位的输入编码转换为 m 位的编码输出，一般情况下 $n < m$ ，输入编码和输出编码之间存在着一一对应的映射关系，每个输入编码产生唯一的一个不同于其他的输出编码。

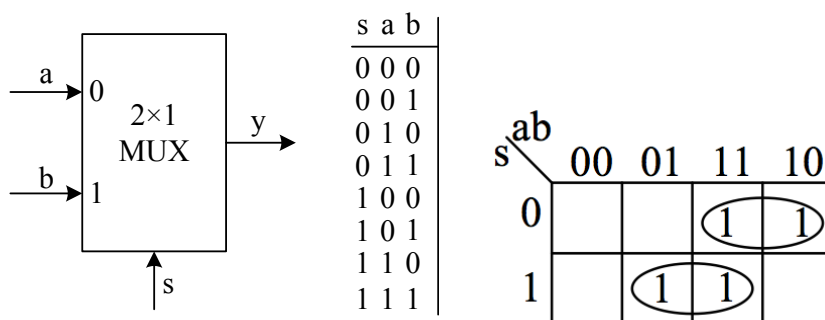
编码器是一种与译码器功能相反的逻辑电路，编码器的输出编码比其输入编码位数少。

本次实验将介绍几种常用的组合逻辑的设计方法；Verilog 语言中的 `always` 语句块、`if-else` 语句和 `case` 语句的使用等。最后请读者自行设计一个多路选择器及 8-3 优先编码器，熟悉电路设计的基本流程和 Vivado 的使用。

1.1 2 选 1 多路选择器

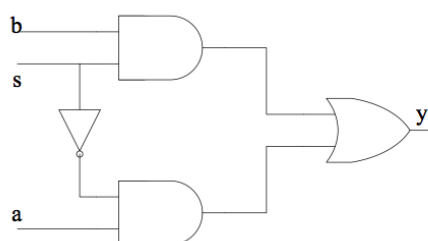
图 1-1(a)是 2 选 1 选择器的模块图和真值表，图中 a 和 b 为输入端； y 为输出端， s 是选择端，选择两个输入的其中一个输出。当 s 为 0 时， y 的输出值为 a 。当 s 为 1 时， y 的输出值为 b 。

图 1-1(b)是 2 选 1 选择器的卡诺图，根据卡诺图可以得出 2 选 1 选择器的表达式为 $y = (\sim s \& a) | (s \& b)$ 。根据表达式画出其逻辑电路如图 1-1(c)所示。



(a) 2选1选择器模块和真值表

(b) 2选1选择器的卡诺图



(c) 2选1选择器的逻辑电路

图 1-1: 2 选 1 选择器

根据图 1-1(c)的逻辑电路图，利用 Verilog HDL 实现 2 选 1 选择器的逻辑电路，如表 1-1所示。

表 1-1: 2 选 1 选择器代码

```

1 module m_mux21(a,b,s,y);
2     input  a,b,s;           // 声明3个wire型输入变量a,b,和s, 其宽度为1位。
3     output y;              // 声明1个wire型输出变量y, 其宽度为1位。
4
5     assign y = (~s&a) | (s&b); // 实现电路的逻辑功能。
6
7 endmodule

```

设计一个激励代码，对设计的选择器的功能进行仿真，如表 1-2所示。

上述代码分析与综合后的仿真结果如图 1-2所示，由图中可以看出，当 $s = 0$ 时， $y = a$ ，即 y 随着 a 值的改变而改变，此时的 b 值无论如何改变都不影响 y 的值。当 $s = 1$ 时， $y = b$ ，即 y 随着 b 值的改变而改变，此时的 a 值无论如何改变都不影响 y 的值。

在 Verilog 中也可以用 if 语句来实现 2 选 1 多路选择器的行为。用 if 语句来设计选择器的程序清单如表 1-3所示：

在 Verilog 中，各语句是并发执行的，模块中所有的 assign 语句、always 语

表 1-2: 2选1 选择器激励代码

```

1  `timescale 10 ns/ 1 ps          // 设置时间尺度和时间精度
2  module m_mux21_vlg_tst();        // 测试代码的端口参数列表为空
3      reg a;                       // 输入变量声明为reg型变量
4      reg b;
5      reg s;
6      wire y;                      // 输出变量声明为wire型变量
7
8      m_mux21 i1 (                 // 对要测试的模块进行实例化
9          .a(a),
10         .b(b),
11         .s(s),
12         .y(y));
13
14     initial begin                 // 初始化输入变量
15         s=0; a=0; b=0; #10;      // 将s, a和b均初始化为“0”，维持10个时间单位
16         b=1; #10;               // 将b改为“1”，s和a的值不变，继续保持“0”，
17                                   // 维持10个时间单位 即10*10ns=100ns
18         a=1; b=0; #10;          // 将a, b分别改为“1”和“0”，s的值不变，
19                                   // 继续保持“0”，维持10个时间单位
20         b=1; #10;               // 将b改为“1”，s和a的值不变，维持10个时间单位
21         s=1; a=0; b=0; #10;     // 将s, a, b分别变为“1,0,0”，维持10个时间单位
22         b=1; #10;
23         a=1; b=0; #10;
24         b=1; #10;
25     end
26 endmodule

```

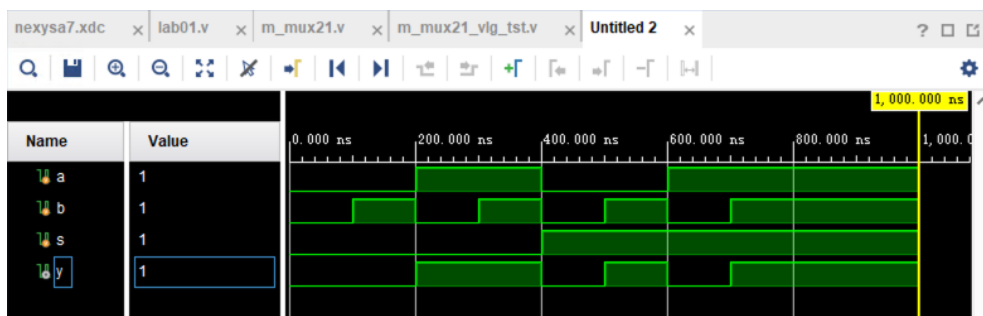


图 1-2: 2选1 选择器仿真结果

句块和实例化语句，其执行顺序不分先后。而 if 语句是顺序执行的语句，其执行过程中必须先判断 if 后的条件，如果满足条件则执行 if 后的语句，否则执行 else 后的语句。Verilog 语法规则，顺序执行的语句必须包含在 always 块中，always 块中的语句按照它们中代码中出现的顺序执行。



Verilog 的语法虽然与 C 语言等高级语言类似，但是请**切记!!**Verilog 语言是

表 1-3: 2 选 1 选择器 if 语句实现

```
1 module mux21b(a,b,s,y);
2     input    a,b,s;
3     output reg y;    // y在always块中被赋值，一定要声明为reg型的变量
4
5     always @ (*)
6         if(s==0)
7             y = a;
8         else
9             y = b;
10 endmodule
```

用来描述电路的语言，不可以高级语言的运行逻辑来指导 Verilog 设计，而是应以描述电路连接的方式来设计。如果能够在写 Verilog 时心中能够画出对应的电路图可以大大减少设计中的错误。例如，上述 if 语句通常可以直接对应到电路中的选择器。

✎ always 语句块的使用

always 块的语句格式如下：

```
always @(< 敏感事件列表 >)
    各可执行的语句;
.....
```

其中敏感事件列表中列出了所有影响 always 块中输出的信号清单，也就是说，如果敏感事件列表中的任何一个变量发生了变化，都要执行 always 语句块中的语句。如 always @ (a or b or s) 表示：只要 a、b、s 中的任何一个变量发生了变化，就立刻执行 always 语句块中的语句。

为了方便起见，敏感列表也可以用 “*” 代替，如 always @ (*), 这里，“*” 号将自动包含 always 语句块中语句或条件表达式右边出现的所有信号。如表 1-3 第 5 行的 always 语句块，只要 always 语句块中表达式右边出现的变量 a 和 b，或者条件表达式中出现的变量 s，这三个变量中的任何一个变量发生了变化，就立刻执行 always 语句块中的语句。

always 语句还有另外一种形式，即：always 后面不带任何有关敏感事件列表的信息，只有 “always” 这个保留字，那么这个时候表明在任何情况下都执行 always 语句块中的语句。

另外，always 块中的输出信号必须被描述成 reg 型，而不是默认的 wire 型。

✎ 关于 if 语句

if 语句是 Verilog HDL 中常用的条件语句，可以和 else 语句配对使用，也可以单独使用。但是，如果 if 语句在使用时没有 else 语句与其配对则会发生这样的情况：编译器判断 if 后面的条件表达式是否满足，如果满足则执行其后的语句，那如果条件表达式不满足呢？这时，编译器就会自动产生一个寄存器来寄存当前的值，在条件不满足时保输出的过去值。这样就会产生用户没有设计的多余的寄存器出来。因此建议读者在使用 if 语句的时候要加上 else 语句与其配对。防止产生多余的寄存器。

另外，编译器默认 if 语句的功能语句只有一条，如果有多条功能语句，要把这些语句用关键词“begin”和“end”将其“括”起来。如：

```
1 if(s==0)
2     y = a;  x = b;
3 else
4     y = b;  x = a;
```

是错误的写法，应改为：

```
1 if(s==0)
2     begin    y = a;  x = b;  end
3 else
4     begin    y = b;  x = a;  end
```

在编程中也可以用条件判断语句代替 if 语句，如果此时不用顺序语句就不需要 always 语句块，比如也可以使用“?”来代替 if 语句，其用法如下：

```
1 assign y = s ? b : a;
```

其含义如下：如果 $s = 1$ ，那么 $y = b$ ；否则 $y = a$ 。则此 2 选 1 选择器代码可另写如下：

表 1-4: 2 选 1 选择器 assign 语句实现

```
1 module mux21c(a,b,s,y);
2     input  a,b,s;
3     output y;    // y不用声明为reg型的了。
4
5     assign y = s ? b : a;
6
7 endmodule
```

1.2 模块实例化

在实验过程中，我们经常需要替换不同实现方式的模块。建议大家对每种方式采用单独的文件和模块名实现。然后，单个实验的顶层 Top 模块提供与 SW 及 LED 接口的连接。在实验时只需要修改顶层模块，调用不同的实现方式即可方便地切换不同的实现。例如，可以用如下方式来组织顶层文件。

```
1 module lab01(
2     input [15:0] SW,
3     output [15:0] LED
4 );
5     m_mux21 a(SW[0],SW[1],SW[15],LED[0]);
6     //mux21b b(SW[0],SW[1],SW[15],LED[0]);
7     //mux21c c(SW[0],SW[1],SW[15],LED[0]);
8 endmodule
```

当然，在输出输出数量足够时，也可以同时实例化多个模块，每个模块用不同的输入输出，这样可以同时进行测试。

 注意：Verilog 的模块实例化与高级语言函数调用有本质区别，实例化更加类似于从芯片库中取出一块芯片，给它起一个名字（实例名），然后连接输入输出引脚。因此，模块实例化同 assign 语句一样，只能在 always 语句块之外进行。

 当需要对某个子模块进行仿真，而非对顶层模块进行仿真时，可以编写对子模块的单独仿真代码，实例化该子模块。但是，仿真时在加入 test bench 之后，需要在 Sources→Simulation Sources→sim_1 下右键点击该 test bench，并将其 **Set as top**。

1.3 4 选 1 多路选择器

4 选 1 多路选择器的模块图和真值表如图 1-3 所示， $a_0 - a_3$ 为 4 个输入端， s_0 和 s_1 是选择端， y 是输出端，根据 s_0 和 s_1 值的不同， y 选择 $a_0 - a_3$ 中的一个输出，具体请见真值表。

Verilog 语言中的 case 语句可以综合出“多路复用器”的电路，它的可读性非常强。表 1-5 所示的是用 case 语句实现 4 选 1 多路选择器的方法。

上述设计的测试代码如表 1-6 所示：

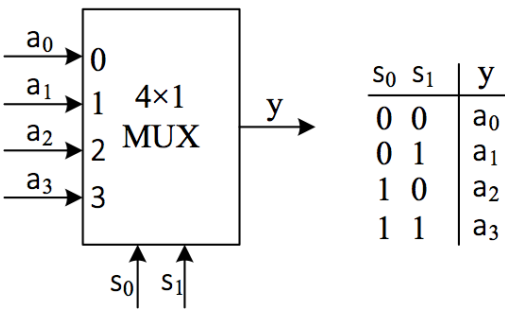


图 1-3: 4 选 1 选择器模块及真值表

表 1-5: 4 选 1 选择器 case 语句实现

```
1 module mux41(a,s,y);
2     input  [3:0] a; // 声明一个wire型输入变量a，其变量宽度是4位的。
3     input  [1:0] s; // 声明一个wire型输入变量s，其变量宽度是2位的。
4     output reg y; // 声明一个1位reg型的输出变量y。
5
6     always @ (s or a)
7         case (s)
8             0: y = a[0];
9             1: y = a[1];
10            2: y = a[2];
11            3: y = a[3];
12            default: y = 1'b0;
13        endcase
14
15 endmodule
```

表 1-5和表 1-6中的程序的仿真图如图 1-4所示。

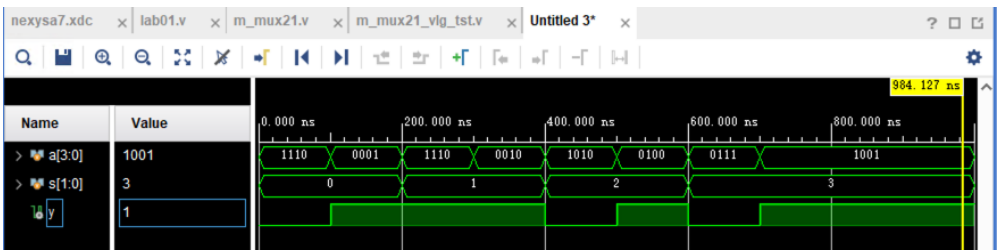


图 1-4: 4 选 1 选择器仿真图

case 语句的使用

case 语句是以关键字 case 和一个被括起来的“选择表达式”开头，表达式の結果表示一个整数。下面是 case 选项，每个选项由选择列表和过程语句构成，选择列表可以是一个整数值，也可以是多个整数值，多个整数值之间以逗号分

表 1-6: 4 选 1 选择器激励代码

```

1  `timescale 10 ns/ 1 ps
2  module mux41_vlg_tst();
3      reg [3:0] a;
4      reg [1:0] s;
5      wire y;
6
7      mux41 i1 (
8          .a(a),
9          .s(s),
10         .y(y));
11     initial begin
12         s=2'b00; a=4'b1110; #10;
13         a=4'b0001; #10;
14         s=2'b01; a=4'b1110; #10;
15         a=4'b0010; #10;
16         s=2'b10; a=4'b1010; #10;
17         a=4'b0100; #10;
18         s=2'b11; a=4'b0111; #10;
19         a=4'b1001; #10;
20     end
21 endmodule

```

开，选择列表和过程语句之间以冒号连接，如：0,1: y = a[0];。

case 语句的执行过程是这样的：先计算出选择表达式的值，在 case 选项中找到和选择表达式值相同的第一个选择，然后执行此选择值后面的过程语句。

case 语句列出的选择列表，有时候不能全部包含选择表达式所有的可能值，这时关键词 default 就要被作为 case 语句的最后一个选项，它表示表达式中那些未被选择列表覆盖的所有其他值。一般情况下即使选择列表列出了选择表达式的所有选项，还是建议保留 default 这一选项。如果选择列表中没有包含选择表达式的所有选项，而此时又没有 default 选项的话，综合器会综合出一个锁存器以保存未被覆盖的情况下输出的过去值。这一般是不希望出现的情况，所以在 case 语句中建议无论如何保留 default 选项。

如果在满足某个表达式值时要执行多条语句，也要用关键词“begin”和“end”将这些语句其“括”起来。

1.4 译码器

常用的二进制译码器（如图 1-5 所示）是一个有 n 路输入和 $m = 2^n$ 路输出的逻辑电路。译码器有一个使能信号 En ，当 $En=0$ 时，无论输入为什么，译码

器没有任何有效值输出；当 $E_n=1$ 时，输入的值决定了输出信号的值。在二进制码中，最常用的输出编码是 $m = 2^n$ 位中取 1 位编码，即任何时刻， m 位输出编码中只能有 1 位有效，其余各位都为 0，这样的二进制编码被称为独热编码（one-hot encoded），意思是那个被置为 1 的码看起来是“热”的，而二进制译码器输出的信号就是独热编码。

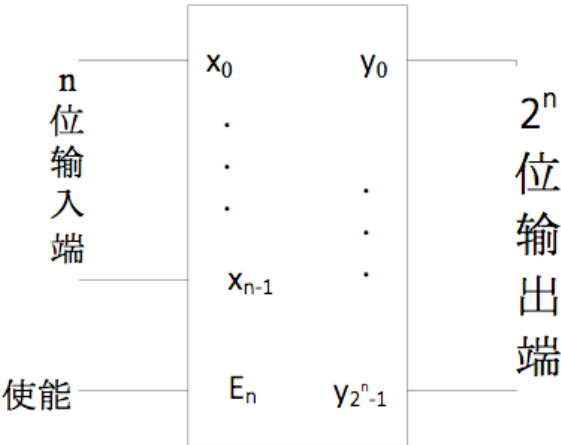


图 1-5: n 位输入， 2^n 位输出的译码器

本次实验首先具体介绍 2-4 译码器和 3-8 译码器的工作原理，学习译码器的设计。

1.4.1 2-4 译码器

2-4 译码器（图 1-6所示），它的输入信号是 x_0 和 x_1 ，对每一个二进制输入信号 x_0 和 x_1 进行译码，译码输出是在 y_0 、 y_1 、 y_2 和 y_3 四位中选择一位使其有效。译码器的输出可以设计成高电平有效或低电平有效，在本例中采用的是高电平有效。

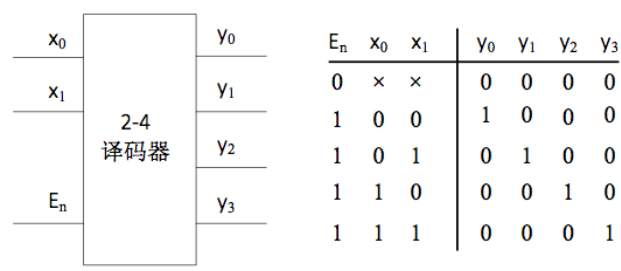


图 1-6: 2-4 译码器

译码器的逻辑电路图如图 1-7所示。

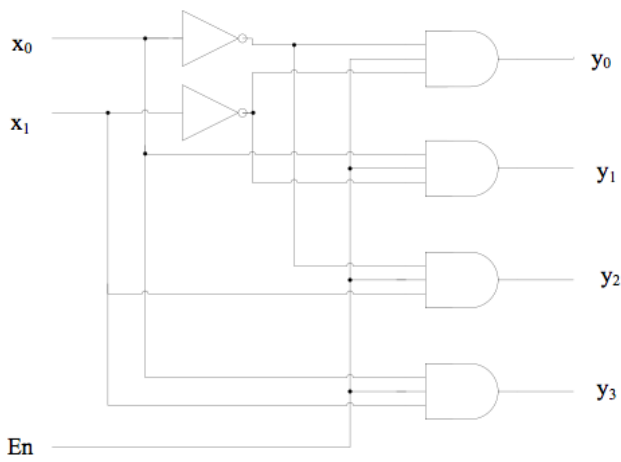


图 1-7: -4 译码器逻辑电路

可以采用硬件描述语言来实现 2-4 译码器电路。表 1-7是一个带有使能端的 2-4 译码器的代码：

程序中用到了数值字符串（literal）“n’Bdd...d”，其中 n 是字符串的位数，用十进制表示，这里字符串的位数是 “dd...d” 这个值存放在机器中（二进制）所用的位数，而不是其用 “B” 进制表示的位数。“B” 是指定基数的单个字母，可以是 b（二进制），o（八进制），d（十进制，可省略）和 h（十六进制）。“dd...d” 是此数值字符串，用 “B” 进制表示的值。

输入设计好的程序代码后，再输入激励代码，对设计进行仿真。

对所设计电路进行功能仿真，如图 1-8所示，分析时序图发现，结果和真值表一致。

表 1-7: 2-4 译码器代码

```

1 module decode24(x,en,y);
2     input  [1:0] x;
3     input  en;
4     output reg [3:0]y;
5
6     always @(x or en)
7         if (en)
8             begin
9                 case (x)
10                    2'd0 : y = 4'b0001;
11                    2'd1 : y = 4'b0010;
12                    2'd2 : y = 4'b0100;
13                    2'd3 : y = 4'b1000;
14                endcase
15            end
16        else y = 4'b0000;
17
18 endmodule

```

表 1-8: 2-4 译码器测试代码

```

1 `timescale 10 ns/ 1 ps
2 module test_decode24();
3     reg  [1:0] x;
4     reg  en;
5     wire [3:0]y;
6     decode24 il (
7         . x (x),
8         . en (en),
9         .y(y) );
10
11     initial
12     begin
13         en = 1'b0; x = 2'b00; #10;
14         x = 2'b01; #10;
15         x = 2'b10; #10;
16         x = 2'b11; #10;
17         en = 1'b1; x = 2'b00; #10;
18         x = 2'b01; #10;
19         x = 2'b10; #10;
20         x = 2'b11; #10;
21     end
22 endmodule

```

1.5 编码器

编码器是一种与译码器功能相反的逻辑电路，编码器的输出编码比其输入编码位数少。

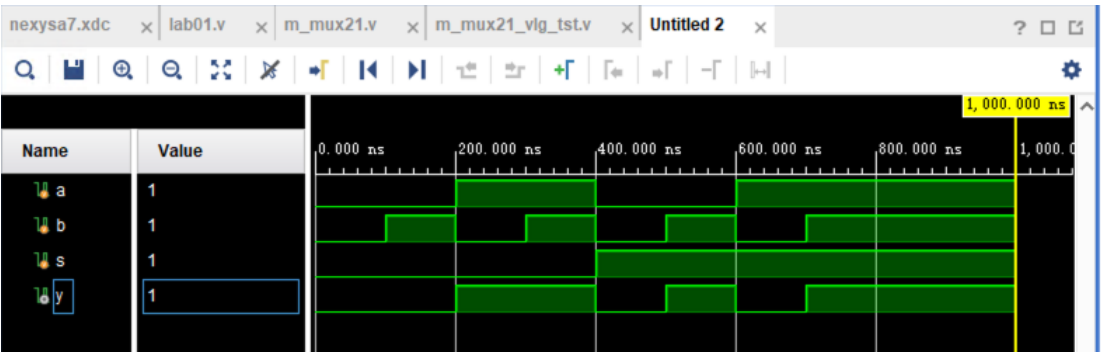


图 1-8: 2-4 译码器仿真结果

常用的二进制编码器把来自于 2^n 条输入线的信息编码转换成 n 位二进制码，如图 1-9 所示。二进制编码器每次输入的 2^n 位信号中只能有一位为 1，其余均为 0（即独热码），编码器的输出端为一个二进制数，用来指示对应的哪一个位输入为 1。

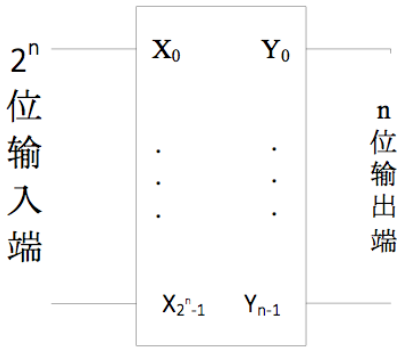


图 1-9: 2^n 位输入， n 位输出的编码器

1.5.1 4-2 编码器

4-2 编码器即为 4 位输入 2 位输出的编码器（如表 1-9 所示），它的输入信号是 x_0 、 x_1 、 x_2 和 x_3 ，输出是 y_0 和 y_1 。本例中采用独热码，每次输入中只有一位为 1，对于有两位或者两位以上为 1 的情况，则将输出置为高阻态。

表 1-10 为该 4-2 编码器的代码。
表 1-11 为该 4-2 编码器的测试代码。
其仿真结果如图 1-10 所示：

表 1-9: 4-2 编码器真值表

x_3	x_2	x_1	x_0	y_1	y_0
0	0	0	1	0	0
0	0	1	0	0	1
0	1	0	0	1	0
1	0	0	0	1	1

表 1-10: 4-2 编码器代码

```
1 module encode42(x,en,y);
2     input  [3:0] x;
3     input  en;
4     output reg [1:0]y;
5
6     always @(x or en)
7         if (en) begin
8             case (x)
9                 4'b0001 : y = 2'b00;
10                4'b0010 : y = 2'b01;
11                4'b0100 : y = 2'b10;
12                4'b1000 : y = 2'b11;
13                default: y = 2'bzz;
14            endcase
15        end
16        else y = 2'bzz;
17
18 endmodule
```

1.5.2 优先编码器

优先编码器允许同时几个输入端有输入信号，即输入不止一个“1”，编码器按输入信号排定的优先顺序，只对同时输入的几个信号中优先权最高的一

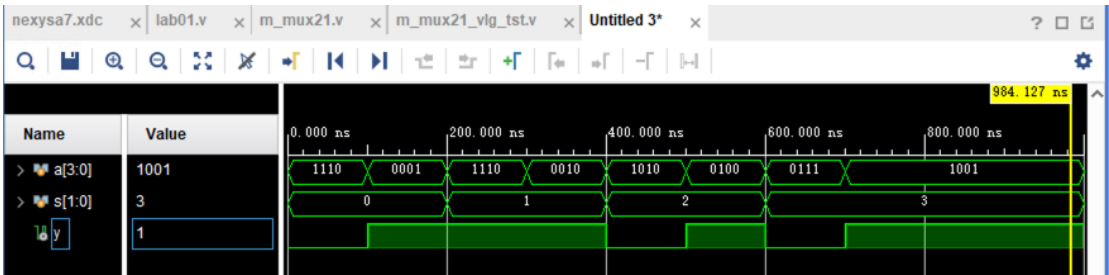


图 1-10: 4-2 编码器仿真结果

表 1-11: 4-2 编码器测试代码

```

1 module test_encode42();
2     reg en;
3     reg [3:0] s;
4     wire [1:0] y;
5
6     encode42 t1(
7         .en(en),
8         .x(s),
9         .y(y));
10    initial
11    begin
12        en=1'b0;  s =4'b0000; #10;
13                  s =4'b0001; #10;
14                  s =4'b0010; #10;
15                  s =4'b0100; #10;
16                  s =4'b1000; #10;
17        en=1'b1;  s =4'b0000; #10;
18                  s =4'b0001; #10;
19                  s =4'b0010; #10;
20                  s =4'b0100; #10;
21                  s =4'b1000; #10;
22    end
23 endmodule

```

个进行编码。

 **casez 与 casex 语句：** case 语句有两种扩展形式 casez 和 casex，他们可以用来处理匹配时需要进行 wildcard 的情况。对于 casez，我们可以用? 或 z 来代表某一个二进制位是无关位，此时无论该位是 0, 1, x 或 z 都将匹配成功。对于 casex，可以用?, z 或 x 来表示无关位。

我们可以利用 casez 语句很方便地实现优先编码器，如表 1-12实现的一个 4-2 优先编码器。其中输入 x 为 4 位，en 为使能端，输出 y 表示 x 的优先编码，高位优先，输出 valid 表示是否 x 有某位输入为 1。

请大家为上述代码编写一段测试代码，并对其进行仿真，查看仿真结果。

表 1-12: 4-2 优先编码器代码

```

1 module priority42(x,en,y,valid);
2     input  [3:0] x;
3     input  en;
4     output reg [1:0]y;
5     output reg valid;
6     integer i;
7     always @(x or en) begin
8         if (en) begin
9             casez(x)
10                4'b1??? : begin y = 2'd3; valid = 1'b1; end
11                4'b01?? : begin y = 2'd2; valid = 1'b1; end
12                4'b001? : begin y = 2'd1; valid = 1'b1; end
13                4'b0001 : begin y = 2'd0; valid = 1'b1; end
14                default : begin y = 2'd0; valid = 1'b0; end
15            endcase
16        end
17        else
18            begin
19                y = 2'd0;  valid = 1'b0;
20            end
21        end
22    endmodule

```

1.5.3 七段数码管

七段 LED 数码管是一种常用的显示元件，常应用于手表、计算器红绿灯等设备中，用于显示数值。图 1-11 是数码管的原理图，数码管分为共阴极和共阳极两种类型，共阴极就是将七个 LED 的阴极连在一起，让其接低电平。这样给任何一个 LED 的另一端高电平，它便能点亮。而共阳极就是将七个 LED 的阳极连在一起，让其接高电平。这样，给任何一个 LED 的另一端低电平，它就能点亮。

Nexys A7-100T 开发平台上有 8 个带小数点的七段数码管，图 1-12 是 Nexys A7-100T 开发平台数据手册中提供的数码管的电路连接图。由电路可以看出，这些数码管是共阳极连接方式，如果给数码管的某个引脚输入低电平则数码管会被点亮，若输入高电平则数码管变暗。

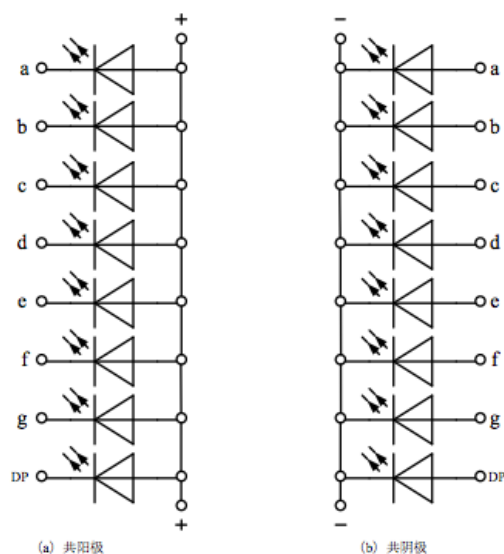
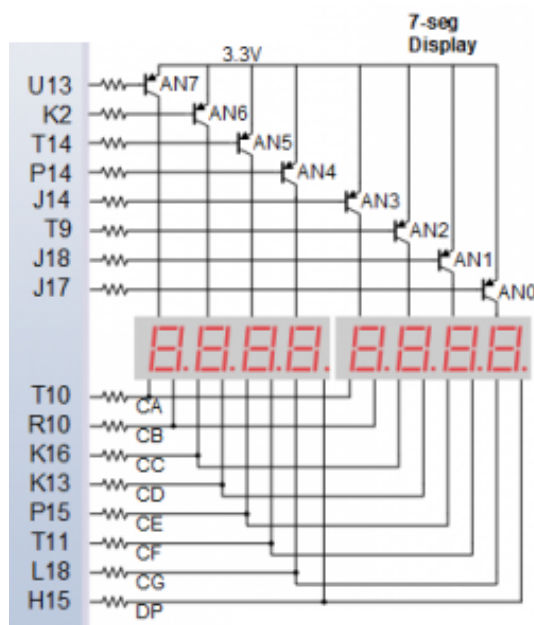


图 1-11: 数码管原理图

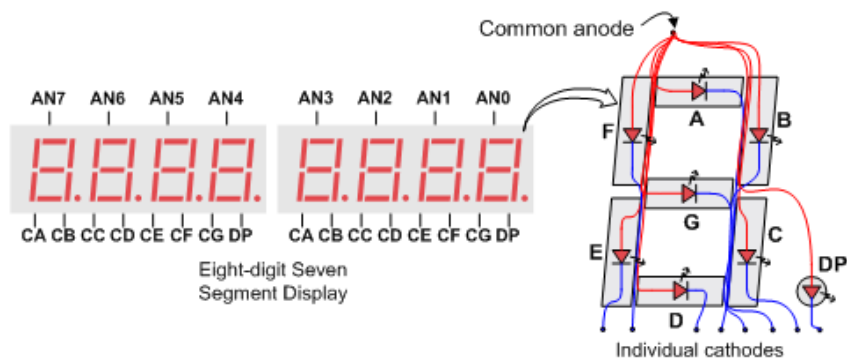
每个数码管都有八个 LED 组成，分别标识为 CA、CB、CC、CD、CE、CF 和 CG，小数点标识为 DP。这些 LED 又和 FPGA 的固定引脚相连，如 CA 和 L3 相连，DP 和 M4 相连（八个数码管的所有 CA 端同时和一个 FPGA 的引脚 L3 相连）。如果在 FPGA 的 L3 端输出一个低电平 0，则数码管的 CA 段将被点亮，FPGA 的这些输出端默认值为 0，如果我们不希望某段数码管点亮，必须从此引脚输入“1”。

仔细分析电路图可以发现，每个数码管的共阳极端相当于是一个数码管的选通端，这八个数码管的共阳极端是通过一个三极管的发射极和高电平接在一起的，因此只有当三极管的基极为低电平时，三极管的共阳极才能正常工作。也就是，如果我们想点亮最右面的数码管，必须要 J17 引脚为低电平“0”（如果不对 J17 进行设置，默认为 0），最右面的数码管相应的需要点亮的段也要输入低电平“0”，此段数码管才会被点亮。这样，我们就可以通过在 U13、K2、T14、P14、J14、T9、J18 和 J17 端输出低电平选中某个数码管，在通过在 T10(CA)、R10(CB)、K16(CC)、K13(CD)、P15(CE)、T11(CF)、L18(CG) 和 H15(DP) 端输出低电平点亮此数码管的某段。

因此，数码管的各个引脚和 FPGA 的相应引脚相连，如果想在最右面的数码管上显示“0”这个字符，那应该从 FPGA 的引脚 N6 输出低电平“0”，选中最右侧的数码管，再使引脚 L6 输出高电平“1”使“G”段为暗，其他各段均为“0”。即从 FPGA 的引脚 N6 输出低电平“0”，再从 T10、R10、K16、K13、



(a) 数码管引脚图



(b) 单个数码管连接图

图 1-12: Nexys A7-100T 数码管连接

P15、T11 和 L18 引脚输出 “7'b1000000” (不包含 DP 时为 7 位, 包含 DP 时为 8 位)。

数字 1~9 及十六进制的 a~f 在数码管上的显示方式如图 1-13 所示。

在我们提供的缺省约束文件中将数码管选择定义为 AN[7:0], 数码管显示分别定义为 CA, CB, ..., CG, DP。为方便起见, 可以把数码管显示也定义成总线形式, 例如 HEX[7:0], 对应修改约束文件和顶层输入输出即可。



注意: 我们在缺省配置文件中添加了很多的引脚, 如果这些引脚在顶层文

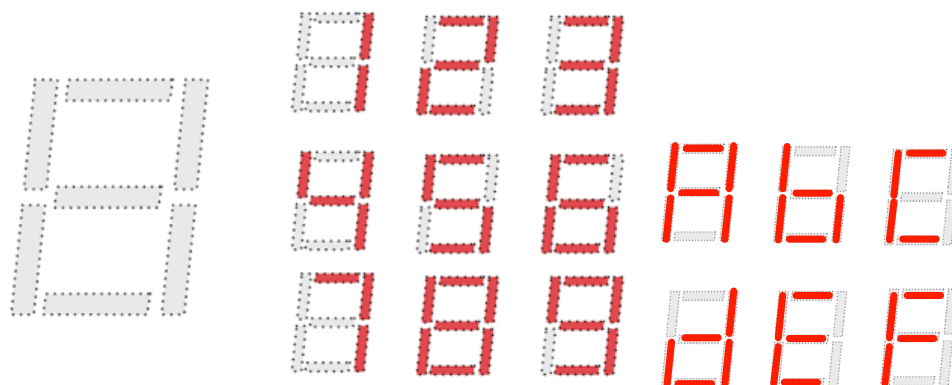


图 1-13: 七段数码管显示方式

件中没有连接，编译时可能会在生成 bitstream 这一步出错。这时，需要添加设置忽略未连接的引脚，方法是建立一个 bit_pre.tcl 文件，并加入以下内容：

```
set_property SEVERITY {Warning} [get_drc_checks UCIO-1]
```

随后在工程设置中的 Bitstream 生成部分添加 prebit 脚本即可，如图 1-14所示：

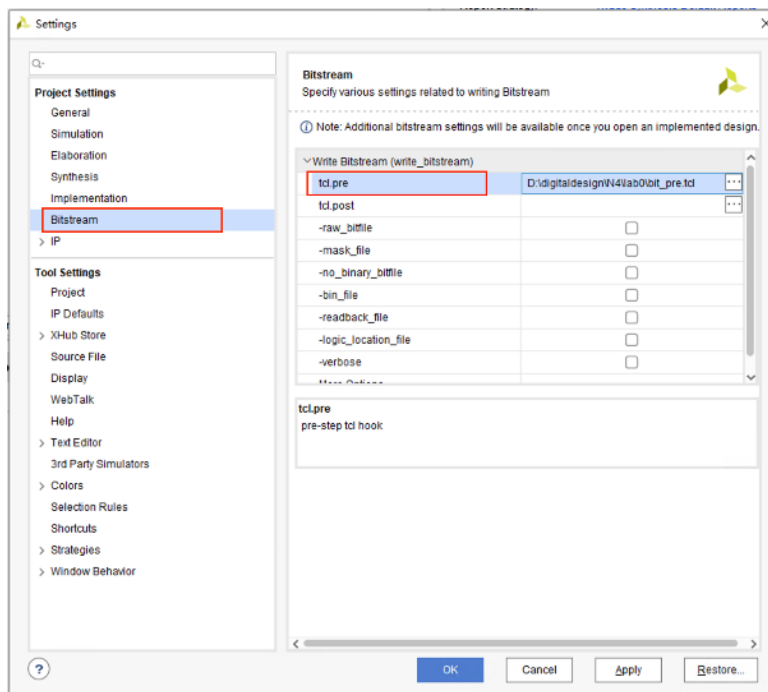


图 1-14: 添加 Bitstream 生成配置

1.6 实验验收内容

1.6.1 上板实验

一、二位四选一选择器

用 case 语句实现一个 2 位 4 选 1 的选择器，如图 1-15 所示，选择器有 5 个 2 位输入端，分别为 X0, X1, X2, X3 和 Y，输出端为 F；X0, X1, X2, X3 是四个 2 位的输入变量。输出 F 端受控制端 Y 的控制，选择其中的一个 X 输出，当 Y = 00 时，输出端输出 X0，即 F = X0；当 Y = 01 时，输出端输出 X1，即 F = X1；以此类推。

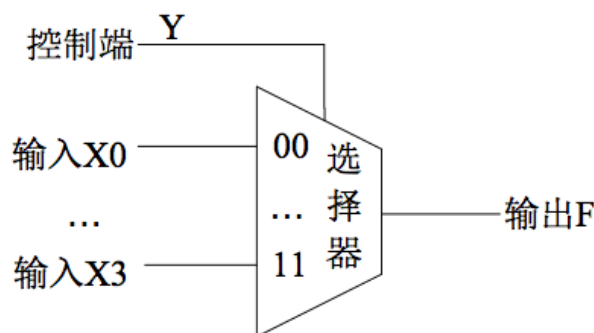


图 1-15: 2 位 4 选 1 选择器

选择开发板上的 SW0 和 SW1 作为控制端 Y，SW2—SW9 作为四个两位数据输入端 X0—X3，将两位的输出端 F 接到发光二极管 LED0 和 LED1 上显示输出，完成设计，对自己的设计进行功能仿真，并下载到开发板上验证电路性能。

二、实现一个 8-3 优先编码器并在七段数码管上显示

功能描述

查找 8-3 优先编码器相关原理和实现方法，实现一个 8-3 编码器，完成 8-3 编码器的设计、功能仿真和硬件实现。

输入一个 8 位二进制数，对此 8 位二进制数进行高位优先编码成一个 3 位二进制值，并根据是否有输入增加一位输入指示位，即 8 个输入全 0 时指示位为 0，有任何一个输入为 1 时指示位为 1。编码器的使能端可选实现。将此编码结果及指示位以二进制形式显示在四个发光二极管 LED 上。再将此结果跟据七

段数码管的显示进行译码，将二进制的优先编码结果以十进制的形式显示在最右边的数码管上。

输入输出建议

输入可以使用拨动开关 SW7-SW0。使能端可以用 SW8。输出为 LED2-0，输出指示可以是 LED4，数码管输出为 AN[7:0] 及 HEX[7:0]。

例：我们从 SW7—SW0 输入 00001110，因为我们设计的是一个高位优先的优先编码器，从左（高位）开始，第一位为 1 的是第 3 号位，那么优先编码器的编码二进制结果就为 011，将这个值显示在发光二极管 LED 上，并且指示位也同时置为 1。再对这个数值根据七段数码管的显示进行译码，此时应显示为“3”。此时我们需要关闭其他数码管，即 AN[7:0] 赋值为“8'b11111110”，同时 HEX[7:0] 赋值为“8'b01100001”，那么在最右边七段数码管上就会显示“3”这个字符。

1.6.2 在线测试

必做 实现一个简单的二位四选一选择器

必做 在线验证优先编码器和数码管显示代码

可选 格雷码